Database Design: Lessons with Images, MCQs and Short-Answer Questions

Companion to the lesson page demo. This document explains how to store lessons, images placed at different positions on the page, and MCQ or short-answer questions. The examples use PostgreSQL, but the model works in MySQL or SQL Server with small changes.

1. Core idea

A lesson is not one big HTML blob. It is an **ordered list of blocks**. Each block has a type (heading, paragraph, image, list, equation, callout, question) and its own content. The page renders blocks in order, so:

- an image is just a block with a **layout** value (`start` , `end` , `wide` , `duo`), which is how the demo places pictures in different positions;
- a question is just a block that points to a row in the `questions` table, so it appears exactly where the author put it;
- reordering content means changing one `position` number, with no text editing.

This keeps content editable from an admin panel, reusable, and easy to render in RTL or LTR.

2. Entities

Table	Purpose
`users`	Students, teachers and admins
`courses`	A subject or grade, for example Grade 7 Biology
`chapters`	Groups lessons inside a course
`lessons`	One lesson page: title, summary, reading time, status
`lesson_blocks`	Ordered content blocks of a lesson
`media_assets`	Uploaded images with alt text, size and caption

Table	Purpose
`questions`	A question: type, prompt, explanation, points
`question_options`	Choices for MCQ questions
`short_answer_keys`	Accepted keywords or model answer for short-answer questions
`attempts`	One student's answer to one question
`lesson_progress`	How far a student has read, and completion status

Relationships: course 1→N chapters 1→N lessons 1→N lesson_blocks. A block of type `image` references one `media_assets` row, and a block of type `question` references one `questions` row. A question has N options (MCQ) or one answer key (short answer). A user has N attempts and N progress rows.

3. Schema (PostgreSQL)

```
CREATE TYPE block_type AS ENUM

('heading','paragraph','list','image','image_group','equation','callout','que
stion');
CREATE TYPE image_layout AS ENUM ('start','end','wide','duo');
CREATE TYPE question_type AS ENUM ('mcq_single','mcq_multi','short_answer');

CREATE TABLE lessons (
  id            BIGSERIAL PRIMARY KEY,
  chapter_id    BIGINT NOT NULL REFERENCES chapters(id) ON DELETE CASCADE,
  slug          TEXT NOT NULL,
  title         TEXT NOT NULL,
  summary       TEXT,
  reading_min   SMALLINT,
  position      INT NOT NULL,
  status        TEXT NOT NULL DEFAULT 'draft',   -- draft | published
  locale        TEXT NOT NULL DEFAULT 'fa',
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (chapter_id, slug)
);
```

```
CREATE TABLE media_assets (
  id          BIGSERIAL PRIMARY KEY,
  url         TEXT NOT NULL,
  alt_text    TEXT NOT NULL,          -- required for accessibility
  caption     TEXT,
  width       INT,
  height      INT,
  mime_type   TEXT,
  uploaded_by BIGINT REFERENCES users(id)
);

CREATE TABLE questions (
  id           BIGSERIAL PRIMARY KEY,
  type         question_type NOT NULL,
  prompt       TEXT NOT NULL,
  explanation  TEXT,                  -- shown after answering
  hint         TEXT,
  points       SMALLINT NOT NULL DEFAULT 1,
  max_length   SMALLINT                -- for short answers
);

CREATE TABLE question_options (
  id          BIGSERIAL PRIMARY KEY,
  question_id BIGINT NOT NULL REFERENCES questions(id) ON DELETE CASCADE,
  label       TEXT NOT NULL,
  is_correct  BOOLEAN NOT NULL DEFAULT false,
  position    SMALLINT NOT NULL
);

CREATE TABLE short_answer_keys (
  question_id  BIGINT PRIMARY KEY REFERENCES questions(id) ON DELETE CASCADE,
  model_answer TEXT,                  -- sample answer shown to the student
  keywords     TEXT[]                 -- optional, for simple auto-checking
);

CREATE TABLE lesson_blocks (
  id          BIGSERIAL PRIMARY KEY,
  lesson_id   BIGINT NOT NULL REFERENCES lessons(id) ON DELETE CASCADE,
  position    INT NOT NULL,
  type        block_type NOT NULL,
  body        TEXT,                    -- heading / paragraph / equation /
callout text
  layout      image_layout,            -- only used by image blocks
```

```
  media_id    BIGINT REFERENCES media_assets(id),
  question_id BIGINT REFERENCES questions(id),
  extra       JSONB NOT NULL DEFAULT '{}',   -- list items, callout title,
image group ids
  UNIQUE (lesson_id, position),
  CHECK (type <> 'image'    OR (media_id IS NOT NULL AND layout IS NOT
NULL)),
  CHECK (type <> 'question' OR question_id IS NOT NULL)
);

CREATE TABLE attempts (
  id            BIGSERIAL PRIMARY KEY,
  user_id       BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  question_id   BIGINT NOT NULL REFERENCES questions(id) ON DELETE CASCADE,
  option_ids    BIGINT[],               -- MCQ selection
  answer_text   TEXT,                   -- short answer
  is_correct    BOOLEAN,                -- NULL until a short answer is
graded
  score         NUMERIC(5,2),
  graded_by     TEXT,                   -- auto | teacher
  submitted_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE lesson_progress (
  user_id     BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  lesson_id   BIGINT NOT NULL REFERENCES lessons(id) ON DELETE CASCADE,
  percent     SMALLINT NOT NULL DEFAULT 0,
  completed_at TIMESTAMPTZ,
  PRIMARY KEY (user_id, lesson_id)
);

CREATE INDEX ON lesson_blocks (lesson_id, position);
CREATE INDEX ON attempts (user_id, question_id, submitted_at DESC);
```

4. How the demo maps to data

On the page	In the database
Figure floated to the right of the text	`lesson_blocks` row, `type='image'`, `layout='start'`

On the page	In the database
Figure floated to the left	`type='image'`, `layout='end'`
Full-width figure	`type='image'`, `layout='wide'`
Two figures side by side	`type='image_group'`, `layout='duo'`, `extra = {"media_ids":[7,8]}`
Equation box, tip box	`type='equation'` and `type='callout'` with text in `body`
MCQ card	`type='question'` pointing to a `questions` row plus `question_options` rows
Short-answer card	`type='question'` pointing to a `questions` row (`type='short_answer'`)

The terms `start` and `end` are used instead of `left` and `right` on purpose. In RTL, `start` is the right side, and in an LTR language it is the left side, so the same data works for both directions. The front end turns `start` and `end` into CSS float values.

5. Loading a whole lesson

One query returns everything needed to render the page in order:

```
SELECT b.position, b.type, b.body, b.layout, b.extra,
       m.url, m.alt_text, m.caption,
       q.id AS question_id, q.type AS question_type, q.prompt, q.hint
FROM lesson_blocks b
LEFT JOIN media_assets m ON m.id = b.media_id
LEFT JOIN questions q    ON q.id = b.question_id
WHERE b.lesson_id = $1
ORDER BY b.position;
```

Fetch the options for the returned question ids in a second query (`WHERE question_id = ANY($1)`). **Never send `is_correct` or `model_answer` to the browser before the student has answered**, otherwise answers can be read from the network tab.

6. Grading rules

- **mcq_single**: compare the chosen `option_ids` with the correct option. Auto-graded, `graded_by='auto'`.
- **mcq_multi**: correct only if the selected set equals the correct set (or give partial credit per option).
- **short_answer**: store `answer_text` with `is_correct = NULL`. Then either match against `keywords` for a quick auto result, or leave it for a teacher, who sets `score` and `graded_by='teacher'`.
- Keep every attempt as a new row instead of overwriting. This gives you history, retries and analytics.

7. Persian and RTL notes

- Use UTF-8 everywhere. In PostgreSQL create the database with a UTF-8 encoding and an ICU collation such as `fa-IR-x-icu` for correct Persian sorting.
- Normalize text before saving: convert Arabic ي and ك to Persian ی and ک, and keep the zero-width non-joiner (U+200C) in words like «می‌شود». Otherwise search will miss matches.
- Store dates as `TIMESTAMPTZ` in UTC and convert to the Jalali calendar only in the interface.
- Store digits as typed. Convert Persian digits to Latin ones for search and numeric columns, and display Persian digits in the UI.
- `lessons.locale` (and optionally `courses.locale`) lets the front end set `dir="rtl"` or `dir="ltr"` per lesson, so English lessons can be added later without changes.

8. Design decisions and alternatives

- **Typed columns plus a small JSONB field.** Frequently queried fields (`type`, `layout`, `media_id`) are real columns with constraints. Rarely queried details (list items, callout titles) go in `extra`. This is easier to validate than all-JSON and more flexible than one table per block type.
- **Questions are separate from blocks.** The same question can be reused in a quiz, an exam or another lesson, and attempts always point to the question rather than to a position on a page.

- **Versioning (optional).** If lessons need an edit history, add `lesson_versions` holding a JSONB snapshot of the blocks, and publish by pointing `lessons.published_version_id` at one version.
- **Soft delete.** Add `deleted_at` to `lessons`, `questions` and `media_assets` if old attempts must stay readable after content is removed.